FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065

(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

1

RANDOM WALK SIMULATION USING AGENTPY

Student Name	Nemeno, Faith S.
Section:	AN43
Professor:	Ms. Crisola Tan
Date:	October 3, 2026

I. BACKGROUND INFORMATION

A **random walk** is a mathematical process that describes a path consisting of a sequence of **random steps**. Random walks are used in many fields, such as physics, biology, economics, and computer science, to model systems that evolve randomly over time.

In this laboratory, we simulate multiple agents moving randomly on a grid using **AgentPy**, a Python library for **agent-based modeling**. Each agent represents an entity that can move one step at a time in a random direction. This simulation helps students understand:

- How **agents move and interact** in a grid-based environment.
- The concept of **bounded movement**, ensuring agents stay within the grid limits.
- What changing **parameters** like the number of agents, grid size, or number of steps affects the system.
- How **agent-based models** can be used to simulate real-world phenomena, such as **diffusion, crowd dynamics, or swarm behavior**.

II. TECHNICAL ASSESSMENT

RANDOM WALK SIMULATION USING AGENTPY

Objective

At the end of this laboratory activity, students are expected to:

1. To understand the concept of **random walks** in a grid environment.
2. To implement a **multi-agent simulation** using Python and AgentPy.
3. To visualize the **movement of agents** on a grid over time.

Materials Needed

- Computer with Python 3.x installed
- Libraries: agentpy, matplotlib
- Code editor (e.g., VSCode, PyCharm, or Jupyter Notebook)

General Instructions

1. This **Technical Assessment (TA4)** must be completed individually.
2. Follow the Task below.
3. Include clear comments in your code to describe its logic.
4. Each program should display the correct input, process, and output.
5. Ensure that your script displays clear and readable outputs in the Command Window.
6. Compile all Python codes and corresponding outputs into one PDF file.
7. Save it as **TA1_lastname.pdf**

   (Example: TA1_Patilano.pdf)
8. Submit the **python file** and **PDF** file.

Activity Instructions

Step 1: Run the Code

1. Open the provided Python code.

Random Walk Simulation (Console-Based)

```
import agentpy as ap
import random
import matplotlib.pyplot as plt
import matplotlib.animation as animation

# -------- Get user input from console --------
num_agents = int(input("Enter number of agents: "))
grid_size = int(input("Enter grid size (e.g., 10 for 10x10): "))
num_steps = int(input("Enter number of steps: "))

# -------- Agent Definition -----------
class RandomWalker(ap.Agent):
    def step(self):
        direction = random.choice([(1,0), (-1,0), (0,1), (0,-1)])
        x, y = self.position
        # Keep agents inside grid
        x = max(0, min(self.model.p.grid_size[0]-1, x + direction[0]))
```

```
        y = max(0, min(self.model.p.grid_size[1]-1, y + direction[1]))
        self.position = (x, y)

# -------- Model Definition -----------
class RandomWalkModel(ap.Model):
    def setup(self):
        # Create agents
        self.agents = ap.AgentList(self, self.p.agents, RandomWalker)

        # Initialize random positions
        for agent in self.agents:
            agent.position = (random.randint(0, self.p.grid_size[0]-1),
                              random.randint(0, self.p.grid_size[1]-1))

        # Create grid (optional)
        self.grid = ap.Grid(self, self.p.grid_size, torus=False)
        self.grid.add_agents(self.agents)

    def step(self):
        for agent in self.agents:
            agent.step()

# -------- Parameters from user input -----------
parameters = {
    'agents': num_agents,
    'grid_size': (grid_size, grid_size),
    'steps': num_steps
}

# -------- Run Model -----------
model = RandomWalkModel(parameters)
model.setup()

# -------- Interactive Animation -----------
fig, ax = plt.subplots()
```

```
ax.set_xlim(0, model.p.grid_size[0])
ax.set_ylim(0, model.p.grid_size[1])
ax.set_xticks(range(model.p.grid_size[0]+1))
ax.set_yticks(range(model.p.grid_size[1]+1))
ax.grid(True)

scat = ax.scatter([], [], s=200)  # s = size of agents

def update(frame):
    model.step()
    x = [agent.position[0] for agent in model.agents]
    y = [agent.position[1] for agent in model.agents]
    scat.set_offsets(list(zip(x, y)))
    return scat,

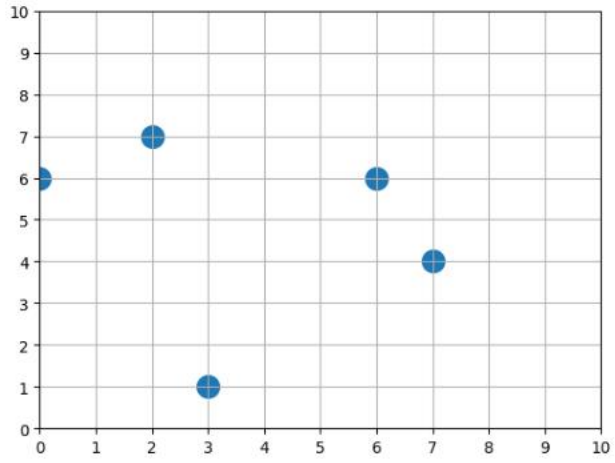
ani = animation.FuncAnimation(fig, update, frames=model.p.steps, blit=True, repeat=False)
plt.show()
```

2. Run the program in your terminal.
3. Enter the following values when prompted:
    - **Number of agents** (e.g., 5)
    - **Grid size** (e.g., 10 for a 10x10 grid)
    - **Number of steps** (e.g., 20)

These values control the number of agents, the size of the environment, and the simulation duration.

Screenshot:

```
Enter number of agents: 5
Enter grid size (e.g., 10 for 10x10): 10
Enter number of steps: 20
```



Step 2: Understand the Agent Behavior

1. Each agent is represented by a dot in the grid.
2. Agents move **randomly** in one of four directions: up, down, left, or right.
3. Agents **cannot leave the grid**; movement is restricted within the grid boundaries.

Step 3: Observe the Simulation

1. After running, a **grid with moving agents** is displayed.
2. Each time step updates the position of all agents.
3. Observe:
    - How agents spread across the grid over time
    - Whether agents cluster in certain areas or move freely

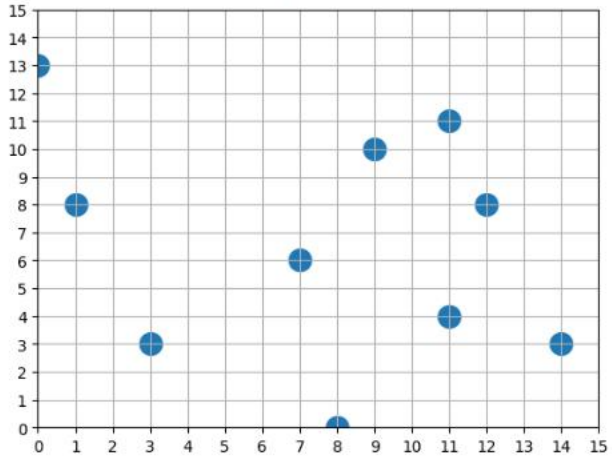
Step 4: Experiment with Parameters

- Repeat the simulation with **different values**:
    - **More agents** (e.g., 10 or 20)
    - **Smaller/larger grid sizes**
    - **Longer simulations** (e.g., 50 steps)

- Observe and record how the **agent movement patterns change**.

Screenshot:

```
Enter number of agents: 10
Enter grid size (e.g., 10 for 10x10): 15
Enter number of steps: 50
```



Laboratory Tasks

1. **Track agent paths**: Modify the code to **record and display the path of each agent** during the simulation.
2. **Change movement rules**: Make agents **avoid each other** or move in a **preferred direction** and observe the resulting patterns.
3. **Analyze final positions**: Record the **final positions** of all agents and analyze their **distribution**.
4. **Compare scenarios**: Run **two simulations with different parameters** and compare the movement patterns.

Modified (Updated Code - Laboratory Tasks) ⇩

```
import random
import agentpy as ap
import matplotlib.pyplot as plt
import matplotlib.animation as animation

# -------- Get user input from console --------
num_agents = int(input("Enter number of agents: "))
grid_size = int(input("Enter grid size (e.g., 10 for 10x10): "))
num_steps = int(input("Enter number of steps: "))

# -------- Agent Definition --------
class RandomWalker(ap.Agent):
    def setup(self):
        # Task 1: list to keep track of everywhere this agent moves
        self.my_path = []

    def step(self):
        x, y = self.position
```

```
        # Task 2: Change movement rules (making agents prefer moving UP or RIGHT)
        # 0 = right, 1 = left, 2 = up, 3 = down
        # added extra choices for up (0,1) and right (1,0) to give them a preferred direction
        possible_moves = [(1, 0), (1, 0), (-1, 0), (0, 1), (0, 1), (0, -1)]
        direction = random.choice(possible_moves)

        # Calculate new spot
        new_x = x + direction[0]
        new_y = y + direction[1]

        # Make sure agents don't walk off the grid borders
        max_limit = self.model.p.grid_size[0] - 1
        x = max(0, min(max_limit, new_x))
        y = max(0, min(max_limit, new_y))

        # Save the updated position and add it to the path history
        self.position = (x, y)
        self.my_path.append(self.position)

# -------- Model Definition --------
class RandomWalkModel(ap.Model):
    def setup(self):
        # Create all the agents
        self.agents = ap.AgentList(self, self.p.agents, RandomWalker)

        # Put agents at random starting spots on the grid
        for agent in self.agents:
            start_x = random.randint(0, self.p.grid_size[0] - 1)
            start_y = random.randint(0, self.p.grid_size[1] - 1)
            agent.position = (start_x, start_y)
            # Add starting position as the first point in their path
            agent.my_path.append(agent.position)
```

```
        # Create grid environment
        self.grid = ap.Grid(self, self.p.grid_size, torus=False)
        self.grid.add_agents(self.agents)

    def step(self):
        # Move each agent every step
        for agent in self.agents:
            agent.step()

# -------- Parameters from user input --------
parameters = {
    'agents': num_agents,
    'grid_size': (grid_size, grid_size),
    'steps': num_steps
}

# -------- Run Model --------
model = RandomWalkModel(parameters)
model.setup()

# -------- Interactive Animation --------
fig, ax = plt.subplots()
ax.set_xlim(-0.5, model.p.grid_size[0] - 0.5)
ax.set_ylim(-0.5, model.p.grid_size[1] - 0.5)
ax.set_xticks(range(model.p.grid_size[0]))
ax.set_yticks(range(model.p.grid_size[1]))
ax.grid(True)

scat = ax.scatter([], [], s=200, color='blue') # blue dots for agents

def update(frame):
    model.step()
    # Get current x and y points for all agents to update plot
    x_coords = [agent.position[0] for agent in model.agents]
```

```
    y_coords = [agent.position[1] for agent in model.agents]
    scat.set_offsets(list(zip(x_coords, y_coords)))
    return scat,

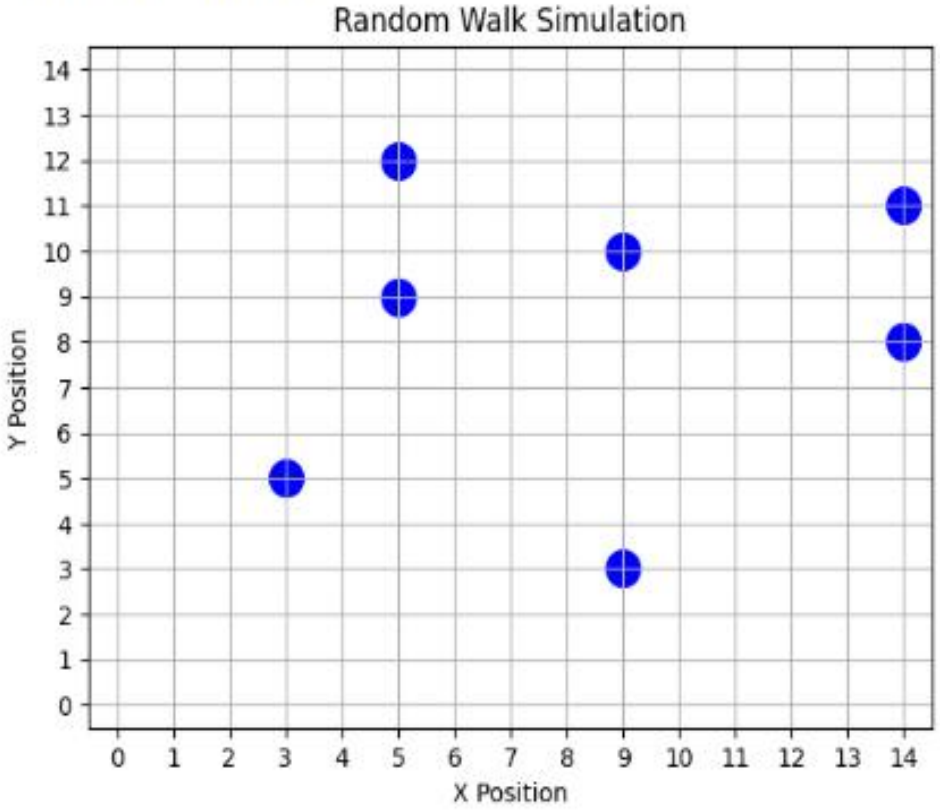
ani = animation.FuncAnimation(
    fig, update, frames=model.p.steps, blit=True, repeat=False
)

plt.title("Random Walk Simulation")
plt.xlabel("X Position")
plt.ylabel("Y Position")
plt.show()

# -------- Task 3: Analyze Final Positions & Paths --------
print("\n--- Final Simulation Results ---")
for i, agent in enumerate(model.agents):
    print(f"Agent {i+1}:")
    print(f"  Final Position: {agent.position}")
    print(f"  Total Steps Taken: {len(agent.my_path)}")
    print(f"  Full Path History: {agent.my_path}\n")
```

Screenshot:

```
Enter number of agents: 7
Enter grid size (e.g., 10 for 10x10): 15
Enter number of steps: 60
```




```
--- Final Simulation Results ---
Agent 1:
  Final Position: (3, 5)
  Total Steps Taken: 3
  Full Path History: [(3, 5), (3, 4), (3, 5)]

Agent 2:
  Final Position: (5, 12)
  Total Steps Taken: 3
  Full Path History: [(6, 13), (6, 12), (5, 12)]

Agent 3:
  Final Position: (9, 3)
  Total Steps Taken: 3
  Full Path History: [(9, 3), (9, 4), (9, 3)]

Agent 4:
  Final Position: (5, 9)
  Total Steps Taken: 3
  Full Path History: [(4, 8), (4, 9), (5, 9)]

Agent 5:
  Final Position: (14, 8)
  Total Steps Taken: 3
  Full Path History: [(14, 7), (14, 8), (14, 8)]

Agent 6:
  Final Position: (14, 11)
  Total Steps Taken: 3
  Full Path History: [(14, 12), (14, 11), (14, 11)]

Agent 7:
  Final Position: (9, 10)
  Total Steps Taken: 3
  Full Path History: [(10, 9), (9, 9), (9, 10)]
```

Step 5: Answer the Questions

1. What happens if the number of agents is **very high** compared to the grid size?

   The grid gets really crowded because there isn't enough space for everyone to spread out. You end up seeing a lot of dots standing on the exact same spot at the same time, which makes the graph look messy and hard to read.

2. How does increasing the number of steps affect the **spread of agents**?

   Giving the agents more steps gives them more time to walk around and explore the map. Instead of staying close to where they started, they end up wandering all over the place and filling in the open areas on the grid.

3. Do the agents ever move outside the grid? Why or why not?

   No, they never go outside the grid. In the code, there is a function using max() and min() that sets invisible walls at the edges. If an agent tries to step past the border, like past X = 9 or Y = 9, the code stops it and keeps it on the grid.

4. How can this simulation be used to **model real-life phenomena** (e.g., diffusion, crowd movement)?

- **Diffusion:** It shows how smoke or perfume molecules start in one spot and randomly bump around until they fill an entire room.
- **Crowd Movement:** It shows how people walk around in busy places like hallways or malls when everyone is picking their own path.
- **Spread of Germs:** It helps us picture how eople bumping into each other randomly in a room can spread sicknesses around.

Answer the Questions⇩